

BÀI TẬP THỰC HÀNH

PHÁT TRIỂN ỨNG DỤNG THIẾT BỊ DI ĐỘNG

5.Introduction to Form, Modal, Redux for React Native

Bài tập 1. Xây dựng ứng dụng quản lý tags – MyTags

Yêu cầu:

1. Frontend
 - 1.1 Theo mẫu bên dưới
 - 1.2 Thiết kế lại giao diện sử dụng phần tử Picker (thư viện @react-native-picker/picker) – (bài tập làm thêm về nhà)
 - 1.3 Gọi API sử dụng hàm map để tạo giao diện theo mẫu
2. Backend
 - Viết API trả về danh sách các tags gồm id và name

❖ Hướng dẫn

Task 1. Tạo mới hoặc mở dự án trên mlearning **mytagsui**

Task 2. Viết **api.js** trả về danh sách các tags gồm id và name (backend)

- Tạo csdl mydb
- Tạo bảng tags gồm cột id và name
- Viết file api.js

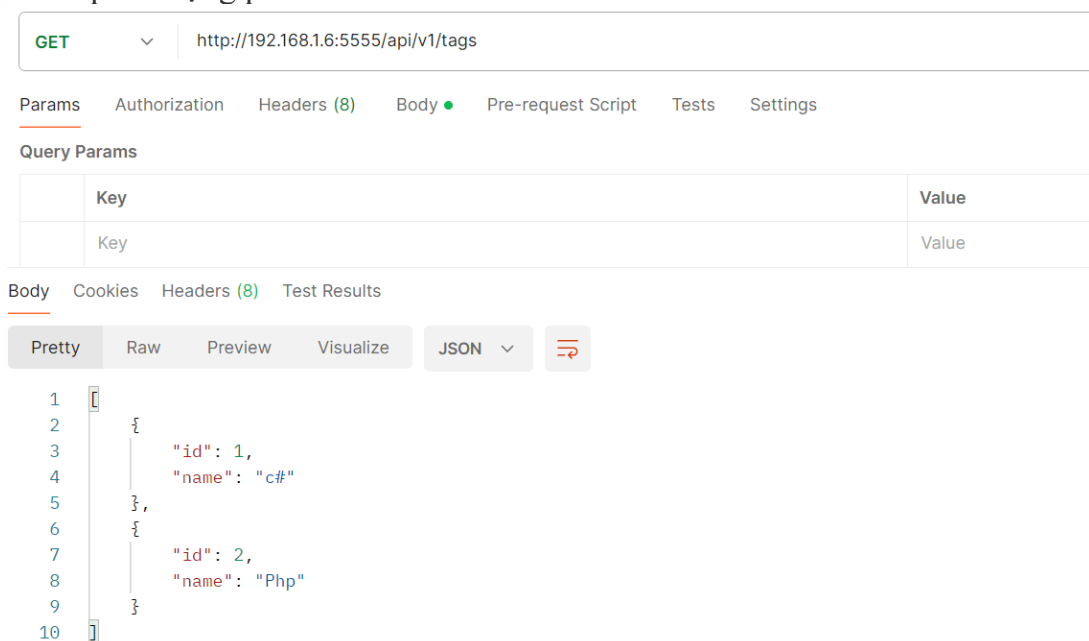
```
var express = require('express');
var bodyParser = require("body-parser");
var cors = require('cors');
var mysql = require('mysql2'); //npm install mysql2
var app = express();
app.use(cors());
app.use(bodyParser.json());
//MYSQL
var con = mysql.createConnection({
  host: "localhost",
  port: "3306",
  user: "root",
  password: "123456",
  insecureAuth: true,
  database: "mydb"
});
// in workbench
//ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY
//'your_password';
con.connect(function (err)
{
  if (err) throw err;
  console.log("Connected!!!")
  var sql = "SELECT * FROM tags";
  con.query(sql, function (err, results)
  {
```

```

        if (err) throw err;
        console.log(results);
    })
});
app.get('/api/v1/tags', function (req, res)
{
    var sql = "SELECT * FROM tags";
    con.query(sql, function (err, results)
    {
        if (err)
        {
            return res.status(500).send(err);
        }
        res.status(200).send(results);
    });
});
//server
var server = app.listen(5555, function ()
{
    var host = server.address().name ? server.address().name : 'localhost';
    var port = server.address().port
    console.log("Example app listening at http://%s:%s", host, port)
})

```

- Test api sử dụng postman



Task 3. Gọi API và sử dụng hàm map để tạo giao diện theo mẫu (frontend)

- Cài đặt thư viện axios
 - `npm install axios`
- Mở và edit code trong file `TagManagementView.js`

```

const ViewAllTagsView = ({ navigation }) =>
{
  // Thay thế dữ liệu json tĩnh với dữ liệu được lấy từ api
  // const items = [
  //   { id: 1, name: 'C#' },
  //   { id: 2, name: 'PHP Laravel' },
  // ]
  // console.log('db', items)
  const [items, setItems] = useState([]);
  const getTags = async () =>
  {
    try
    {
      const res = await axios.get('http://localhost:5555/api/v1/tags');
      console.log(res.data);
      setItems(res.data)
    } catch (err)
    {
      console.error(err);
    }
  };

  useEffect(() =>
  {
    getTags()
  }, [])

  return (
    <View style={styles.crudContainer}>
      <View style={styles.listTagsContainer}>
        {items != null ? (
          items.map((item) =>
            {
              return (
                <View key={item.id}>
                  <TouchableOpacity
                    style={styles.tagContainer}
                  >
                    <Text style={styles.itemText}>{item.name}</Text>
                  </TouchableOpacity>
                </View>
              );
            })
          ) : (
            <View>No tags</View>
          )}
      </View>
      <TouchableOpacity
        style={styles.btn}
        onPress={() =>

```

```

    {
      navigation.navigate("Create");
    }
  }
  >
  <Text style={styles.btnTitle}>Add New </Text>
</TouchableOpacity>

</View>
);
};
export default TagManagementView;

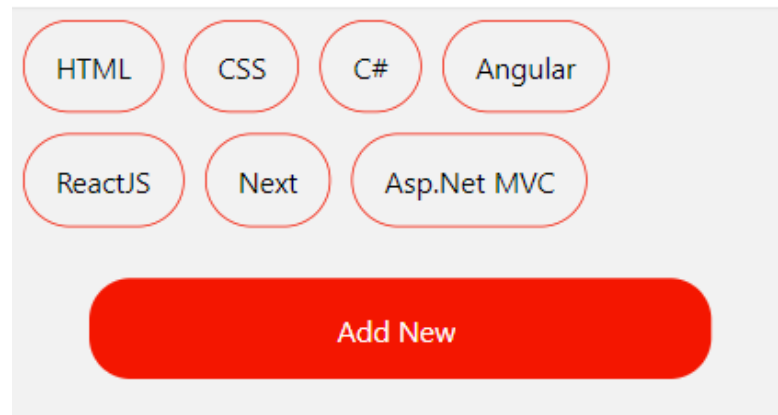
```

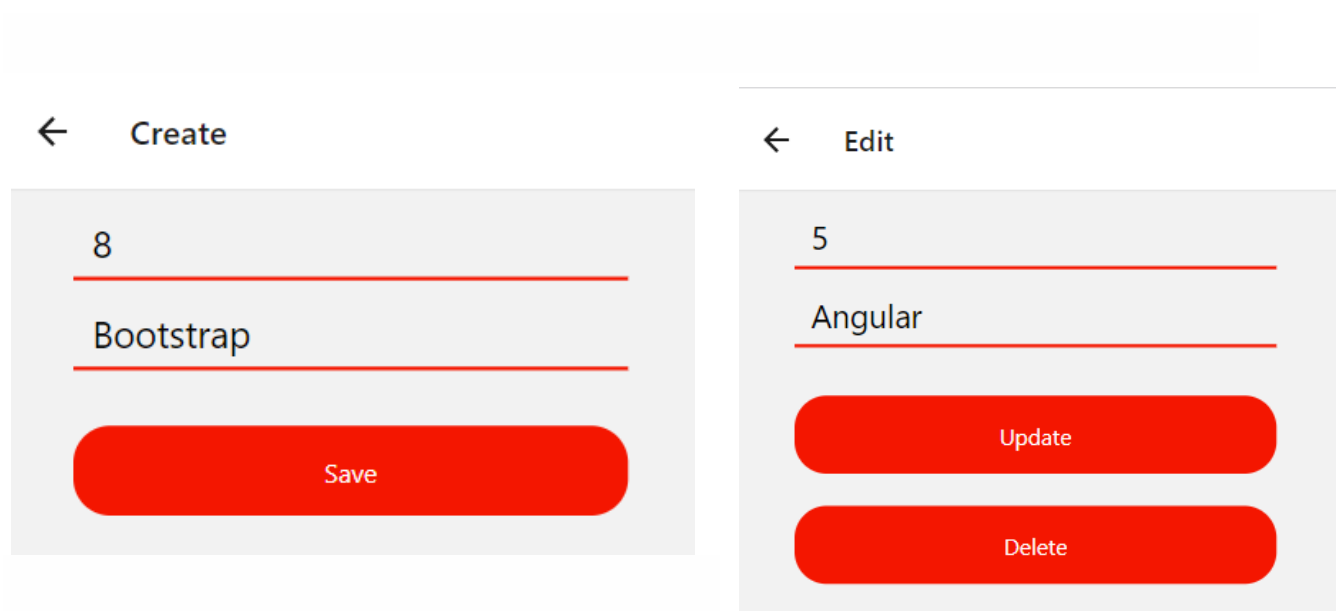
Bài tập 2. Xây dựng ứng dụng quản lý tags – MyTags

Yêu cầu:

- Backend (CRUD) – tạo các api thực hiện tính năng thêm mới / cập nhật hoặc xóa tag theo id
 - Viết api/v1/createNewTags
 - Viết api/v1/updateTag/{id}
 - Viết api/v1/deleteTag/{id}
 - Viết api/v1/findTagById/{id}
- Frontend – tạo giao diện gọi đến các api thực hiện tính năng thêm mới / cập nhật hoặc xóa tag theo id

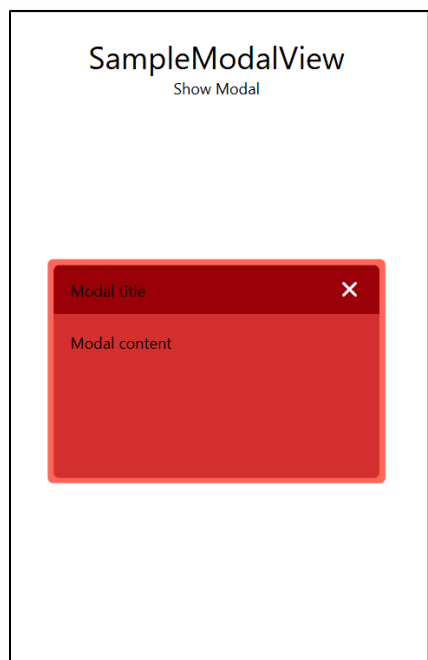
ViewAll





Bài tập 3. Thiết kế component

Khi click vào “Show Modal” sẽ popup modal đơn giản như sau:



Hướng dẫn

Bước 1. Create a new expo app

- **npx create-expo-app my-app --template blank@54**
- **cd my-app**

Bước 2. Start project

- **npx expo start**

Bước 3. Chạy thử nghiệm ứng dụng với Android simulator



Bước 4. Tạo file ./components/SampleModalView.js và copy code sau:

```
import { Icons } from '@expo/vector-icons';
import React, { useState } from 'react';
import { Text, View, StyleSheet, Modal, TouchableOpacity, Pressable,
ImageBackground, Image } from 'react-native';

const SampleModalView = ({ params, }) =>
{
  const [modalShow, setModalShow] = useState(false);

  return (
    <View style={styles.container}>
      <View style={styles.topContainer}>
        <Text style={styles.titleText}>SampleModalView</Text>
        <TouchableOpacity onPress={() => setModalShow(true)}>
          <Text>Show Modal</Text>
        </TouchableOpacity>
      </View>

      <Modal
        transparent={true}
        visible={modalShow}
        animationType={'slide'}
        onRequestClose={() => setModalShow(false)}>
        <Pressable style={styles.modalContainer} onPress={(evt) =>
evt.target == evt.currentTarget ? setModalShow(false) :
setModalShow(true)}>

          <View style={styles.box}>
            <View style={styles.boxHeader}>
              <Text style={{ flexGrow: 1 }}>Modal
title</Text>
```

```

                                <Icons name='close' size={24}
color={'#FFF'} />

                                </View>
                                <View style={styles.boxContent}>
                                    <Text>Modal content</Text>
                                </View>
                            </View>

                            </Pressable>
                        </Modal>
                    </View>
                );
            }

export default SampleModalView;

const styles = StyleSheet.create({
  container: {
    flex: 1,
    //alignItems:"center",
  },
  topContainer: {
    flex: 1,
    alignItems: "center",
  },
  bottomContainer: {
    flex: 6,
    //backgroundColor:'#FF0000',
    alignItems: "center",
  },
  titleText: {
    fontSize: 28,
  },
  modalContainer: {
    width: '100%',
    height: 370,
    marginTop: 200,
    padding: 20,
    alignItems: 'center',
    //backgroundColor: "rgba(1,1,1,0.5)"
  },
  btnClose: {
    width: 30,
    height: 30,
    borderRadius: 30 / 2,
    backgroundColor: 'orange',
    alignItems: 'center',
  },
});

```

```

        justifyContent: 'center',
        marginLeft: 250,
        zIndex: 10
      },
      box: {
        width: '90%',
        height: 200,
        backgroundColor: '#ff6659',
        borderRadius: 5,
        padding: 5,
      },
      boxHeader: {
        flex: 1,
        flexDirection: 'row',
        backgroundColor: '#9a0007',
        borderTopLeftRadius: 5,
        borderTopRightRadius: 5,
        padding: 15,

        alignItems: 'center',
      },
      boxContent: {
        flex: 8,
        backgroundColor: '#d32f2f',
        borderBottomLeftRadius: 5,
        borderBottomRightRadius: 5,
        padding: 15,
      }
    }
  })

```

Bước 5. Edit code file App.js gọi component SampleModalView

```

import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';
import TagManagementView from './components/TagManagementView';
import SampleModalView from './components/SampleModalView';

export default function App()
{
  return (
    <View style={styles.container}>
      <SampleModalView />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'white',
    padding: 20
  },

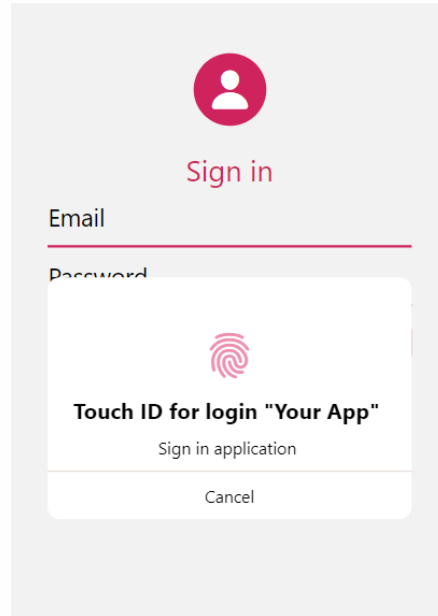
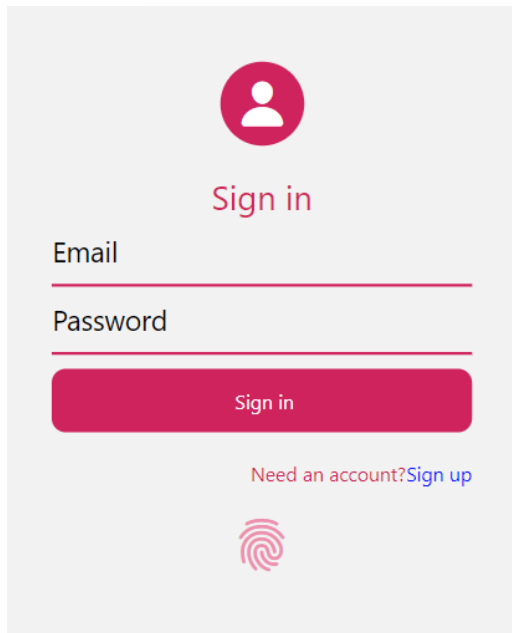
```


});

Chạy ứng dụng với Simulator và thiết bị vật lý (Ios hoặc Android)

Bài tập 4. Thiết kế Modal popup sau:

- Khi người dùng click chọn đăng nhập bằng icon vân tay



- Popup hộp thoại quảng cáo

